

Implementing 2-dimensional physics for browser based games using Box2dWeb

>>Here's how you go about building the next Angry Birds<<

Introduction

We've all had that phase where we've spent countless hours playing browser-based games. Back in the day it was mostly recreations of classics like Hangman and Tetris from the bygone era of MS-DOS. The evolution of games soon brought into existence much improved graphical fidelity but realism, in terms of physics was just being conceived. We moved away from all of that till Social Networking websites dragged us right back into browser-based gaming. In the beginning it was all just Flash with ActionScript but with improved HTML standards, we started seeing JavaScript physics libraries becoming more and more prominent and Flash being put on the backburner.

Soon enough we had games like Crazy Taxi, Angry Birds and the whole lot coming into existence. Some of these seemed to have a dash of physics simulation added while some games' entire USP relied simply on physics. We're of course talking about Angry Birds. It has spawned thousands of clones and we slowly started seeing more physics-based games breaking the surface. Now that HTML5 has shown a marked improvement, with more features being made native, the memory overhead and complexity of coding for the web came down. We even have intermediate programming languages like asm.js which happens to be a subset of JavaScript, hence, it runs on all browsers natively. Games that were written for C/C++, Lua, Qt, etc. can all be ported into asm.js using a source-to-source compiler. This can be played on any platform that exists now and all you need is an internet connection.

What makes games feel real is the physics in it. Angry Birds wouldn't feel as good, and definitely wouldn't have achieved the level of popularity that it has, if there had been no physics in the game. The piggies rolling over, bumping into structures and bringing down entire wooden buildings feel so natural. This is made possible thanks to a Physics engine. Mainstream games use much advanced APIs like PhysX and Havok. Browser based games use JavaScript libraries like box2dweb, Ammo.js, Cannon.js, JigLibJSetc. Box2D is an open source 2D physics engine that has been used for implementing 2D game physics across a great many platforms. Some of the games built using this engine are Angry Birds, Happy Wheels, Tiny Wings, Crayon Physics Deluxe, Limbo etc. Box2D has been used on Nintendo DS, Wii, mobile OSes like Android, BB10 and iOS. It has also been ported to many languages and we're going to looking at one such port – Box2dWeb, a JavaScript port of Box2D.

Why Box2dWeb?

Firstly, we're only working with 2-dimensional physics and



Using Box2DWeb can be just as easy as this

Box2dWeb is only programmed to simulate physics in a 2-dimensional world. Its constraint system is called joints and its quite easy to setup and use. Even setting up collision detection is relatively simple since you just have to add a listener to that particular object. It is quite fast to execute and as with any programming language, only poor coding can mess that up. The API is very simple, you can always refer the original C/C++ manual or the manual for the Flash port if you find the JavaScript manual lacking. Overall, it's a feature rich 2-dimensional physics engine that is supported on all major browsers.

Setting up a simple scene

We'll get the body tag out of the way first. It's just the canvas within which the demo takes place.

```
<body onload="init();">
  <canvas id="canvas" width="600" height="400"></canvas>
</body>
```

Now, the very first thing that needs to be done is to set up a world in which all of the actions are to take place. To do this we need to create an instance of b2World which in turn requires you to declare a vector along which the force of gravity will act upon all objects (both free-falling and pivoted) within the canvas. Vectors are defined using the b2Vec2 function, which is a 2D column vector with two elements.